

KidsEdu AI — Персональная образовательная видео-платформа с ИИ-куратором

SPEC.md / CLAUDE.md — Полная техническая спецификация проекта

1. Концепция и цель проекта

KidsEdu AI — это автономная экосистема фильтрации, планирования и воспроизведения образовательного видеоконтента для двух детских профилей (возраст: **2 года и 5 лет**), разворачиваемая на выделенных смартфонах Samsung Galaxy S20+ в защищенном режиме киоска (Kiosk Mode).

Ключевые задачи системы:

- **Абсолютная изоляция от алгоритмов рекомендаций:** Полное исключение реакций YouTube, VK Видео, дофаминового мусора (шортсы, распаковки, громкий треш-контент, скрытая реклама).
 - **Детерминированный годовой учебный план:** Автоматический отбор видеоконтента на основе педагогических программ для дошкольников с динамической адаптацией под успеваемость.
 - **Геймифицированный дофаминовый ограничитель:** Внедрение обязательных интерактивных микро-квизов с голосовым сопровождением между ролика, блокирующих непрерывное бессознательное потребление.
 - **Прозрачность для родителей:** Веб-панель мониторинга и Telegram-бот для управления расписанием, просмотра аналитики и оперативной корректировки тем.
-

2. Целевая аудитория и профилизация

Параметр	Профиль «Младшая» (2 года)	Профиль «Старшая» (5 лет)
Образовательный фокус	Базовые цвета, геометрия, звуки природы/животных,	Чтение/фонетика, счет и логика, устройство окружающего мира,

Параметр	Профиль «Младшая» (2 года)	Профиль «Старшая» (5 лет)
	развитие речи, крупная моторика, эмоциональный интеллект.	основы космоса и физики, английский язык (A0), социализация.
Длительность сессии	10–15 минут (1–2 коротких видео по 3–5 мин).	20–30 минут (2–3 видео по 7–10 мин).
Формат микро-тестов	Голосовой вопрос (TTS) + выбор из 2 крупных визуальных карточек (без текста).	Голосовой + текстовый вопрос + выбор из 3–4 иллюстрированных карточек .
Интерфейс плеера	Сверхкрупные элементы (>120px), отсутствие мелких текстовых кнопок.	Крупные элементы, цветовая индикация прогресса темы недели.
Лимит времени	До 30 минут в день.	До 60 минут в день.



3. Технический стек и инфраструктура

Система строится исключительно на **бесплатных, открытых и Self-Hosted** решениях для обеспечения полной автономности.

Backend & Core Services

- **Среда выполнения:** Windows 11 + WSL2 (Ubuntu 22.04 LTS) / Docker Compose.
- **Язык разработки:** Python 3.11+.
- **Веб-фреймворк:** FastAPI (асинхронный обработчик API, OpenAPI 3.0).
- **Валидация данных:** Pydantic v2 (строгая валидация схем и Structured Outputs для LLM).
- **ORM & БД:** SQLAlchemy 2.0 (AsyncIO) + SQLite (с поддержкой WAL-режима для локального запуска) или PostgreSQL 16.
- **Парсинг и медиа-потоки:** yt-dlp (извлечение субтитров `.vtt/.srt`, метаданных и прямых HLS/MP4 ссылок).
- **Фоновые задачи:** APScheduler (периодический парсинг, сбор статистика и очистка кеша).

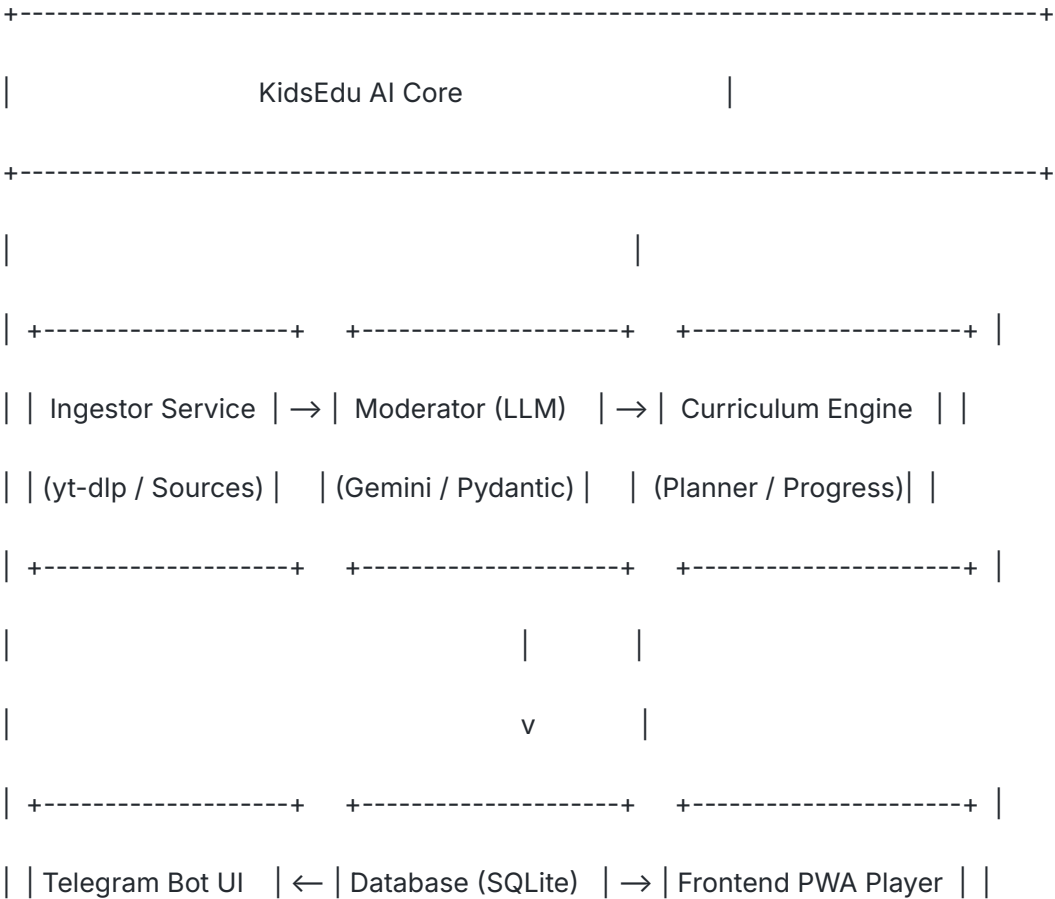
AI & Media Engines

- **LLM-маршрутизатор:** Google Gemini API (`gemini-2.5-flash` / `gemini-2.5-pro` бесплатный тип) через `google-genai` SDK для анализа субтитров и генерации вопросов с Pydantic Schema Enforcement.
- **Синтез речи (TTS):** `edge-tts` (Python-библиотека для бесплатной генерации четкого русского текста в аудио MP3 через Microsoft Edge Read Aloud API).
- **Бот-управление:** `aiogram 3.x` (Telegram Bot API с поддержкой Inline App и WebApp).

Frontend & Device Control

- **Детский плеер & Admin UI:** React 18 / Vite, TailwindCSS, Lucide-Icons (PWA-адаптация под разрешение 20:9 экрана Samsung Galaxy S20+ — 3200×1440 / Viewport 800×360px).
- **Защита устройства:** Fully Kiosk Browser Android app (режим блокировки выхода, скрывание системных баров status/navigation bar, запрет вызова настроек, автозапуск PWA).

4. Подробная архитектура модулей



```
| | (Parent Control) | | (SQLAlchemy Async) | | (Kiosk + Quiz Overlay) |
| +-----+ +-----+ +-----+ |
+-----+
```

Модуль 1: Контент-парсер и ИИ-модератор (`app/services/ingestor.py` & `moderator.py`)

1. **Белый список каналов (Seed List):** Конфигурируемый перечень проверенных каналов и плейлистов YouTube и VK Видео.
2. **Сбор и экстракция:** Фоновый воркер через `yt-dlp` каждые 12 часов запрашивает свежие ролики, извлекает текстовые субтитры и метаданные.
3. **Строгий ИИ-фильтр:** Субтитры отправляются в Gemini 2.5 Flash с требованием строгого ответа по JSON-схеме.
4. **Критерии отклонения контента:**
 - Наличие резких вскриков, визгов, агрессивных звуковых эффектов.
 - Контент типа «Unboxing», реклама игрушек, демонстрация сладостей.
 - Быстрая смена кадров (клиповый монтаж < 1.5 сек на кадр).
 - Отсутствие явной образовательной цели.
5. **Генерация вопросов:** При одобрении ролика (`is_approved = True`) LLM автоматически генерирует 2 проверочных квиса с привязкой к таймкоду конца видео.

Модуль 2: Генератор годового учебного плана (`app/services/curriculum.py`)

1. **Иерархия плана:**
Годовой план $\rightarrow$ Модуль (Месяц) $\rightarrow$ Тема (Неделя)
 $\rightarrow$ Дневной плейлист (1-3 видео).
2. **Динамическая сложность:**
 - При результатах квисов **>80% успеха** за неделю система повышает сложность темы или переводит на следующий модуль.
 - При результатах **<50% успеха** система подбирает дублирующие/закрепляющие ролики по той же теме с другим визуальным рядом.

Модуль 3: Защищенный плеер и движок квисов (`frontend/src/pages/ChildPlayer.tsx`)

1. **Изоляция видеоплеера:** Использование HTML5 / YouTube IFrame API с атрибутами `controls=0, rel=0, disablekb=1`. Поверх iframe накладывается прозрачный блок, перехватывающий клики, исключая возможность перехода на внешние сайты.

2. **Квиз-пауза:** По завершении ролика видео скрывается, включает компонент `QuizOverlay.tsx`.
3. **Озвучка через TTS:** Сервис `tts.py` заранее синтезирует `.mp3` файл вопроса через `edge-tts`. Плеер проигрывает его при открытии квиза.
4. **Геймифицированная обратная связь:** При правильном ответе проигрывается мягкий анимационный эффект и начисляются звездочки. При ошибке подсвечивается верный вариант без негативных штрафов.

Модуль 4: Родительский контроль и аналитика (`app/bot/ & frontend/src/pages/ParentAdmin.tsx`)

1. **Дашборд:** Просмотр общего времени просмотра, списка изученных навыков, точности ответов на квизы.
 2. **Telegram-управление:**
 - `/status` — текущий статус устройств и время просмотра за сегодня.
 - `/add_topic <тема>` — принудительно добавить тему в план текущей недели (например, «гигиена зубов»).
 - `/block_video <id>` — моментально исключить ролик из базы.
-

5. Схема базы данных и Pydantic-модели

SQLAlchemy Async Схема (`app/models/database.py`)

```
import enum
```

```
from datetime import datetime
```

```
from typing import Optional, List
```

```
from sqlalchemy import String, Integer, Boolean, Float, ForeignKey, DateTime, Text, JSON, Enum
```

```
from sqlalchemy.orm import DeclarativeBase, Mapped, mapped_column, relationship
```

```
class Base(DeclarativeBase):
```

```
    pass
```

```
class PlatformEnum(str, enum.Enum):
```

```
    YOUTUBE = "youtube"
```

```
VK = "vk"
```

```
class CurriculumStatusEnum(str, enum.Enum):
```

```
    PLANNED = "planned"
```

```
    ACTIVE = "active"
```

```
    MASTERED = "mastered"
```

```
    NEEDS_REVIEW = "needs_review"
```

```
class ChildProfile(Base):
```

```
    __tablename__ = "child_profiles"
```

```
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
```

```
    name: Mapped[str] = mapped_column(String(50))
```

```
    age: Mapped[int] = mapped_column(Integer) # 2 или 5
```

```
    daily_limit_minutes: Mapped[int] = mapped_column(Integer, default=30)
```

```
    avatar_url: Mapped[Optional[str]] = mapped_column(String(255))
```

```
    created_at: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)
```

```
    curriculum_nodes: Mapped[List["CurriculumNode"]] = relationship(back_populates="child")
```

```
    watch_logs: Mapped[List["WatchLog"]] = relationship(back_populates="child")
```

```
class CurriculumNode(Base):
```

```
    __tablename__ = "curriculum_nodes"
```

```
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
```

```
    child_id: Mapped[int] = mapped_column(ForeignKey("child_profiles.id"))
```

```
    month: Mapped[int] = mapped_column(Integer)
```

```
    week: Mapped[int] = mapped_column(Integer)
```

```
    topic: Mapped[str] = mapped_column(String(100))
```

```
target_skills: Mapped[dict] = mapped_column(JSON) # ["colors", "counting"]
```

```
status: Mapped[CurriculumStatusEnum] = mapped_column(Enum(CurriculumStatusEnum),  
default=CurriculumStatusEnum.PLANNED)
```

```
child: Mapped["ChildProfile"] = relationship(back_populates="curriculum_nodes")
```

```
class VideoItem(Base):
```

```
    __tablename__ = "video_items"
```

```
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
```

```
    source_platform: Mapped[PlatformEnum] = mapped_column(Enum(PlatformEnum))
```

```
    external_id: Mapped[str] = mapped_column(String(100), unique=True, index=True)
```

```
    title: Mapped[str] = mapped_column(String(255))
```

```
    duration_sec: Mapped[int] = mapped_column(Integer)
```

```
    transcript_text: Mapped[Optional[str]] = mapped_column(Text)
```

```
    transcript_summary: Mapped[Optional[str]] = mapped_column(Text)
```

```
    age_group: Mapped[int] = mapped_column(Integer) # 2 или 5
```

```
    topic_tags: Mapped[dict] = mapped_column(JSON) # ["shapes", "animals"]
```

```
    educational_score: Mapped[int] = mapped_column(Integer) # 1-10
```

```
    is_approved: Mapped[bool] = mapped_column(Boolean, default=False)
```

```
    rejection_reason: Mapped[Optional[str]] = mapped_column(String(255))
```

```
    stream_url_cache: Mapped[Optional[str]] = mapped_column(Text)
```

```
    quizzes: Mapped[List["QuizQuestion"]] = relationship(back_populates="video")
```

```
class QuizQuestion(Base):
```

```
    __tablename__ = "quiz_questions"
```

```
    id: Mapped[int] = mapped_column(Integer, primary_key=True)
```

```

video_id: Mapped[int] = mapped_column(ForeignKey("video_items.id"))

question_text: Mapped[str] = mapped_column(Text)

audio_tts_path: Mapped[Optional[str]] = mapped_column(String(255))

options_json: Mapped[dict] = mapped_column(JSON) # [{"label": "...", "emoji": "🍌",
"is_correct": true}]

correct_option_index: Mapped[int] = mapped_column(Integer)

video: Mapped["VideoItem"] = relationship(back_populates="quizzes")

class WatchLog(Base):

    __tablename__ = "watch_logs"

    id: Mapped[int] = mapped_column(Integer, primary_key=True)

    child_id: Mapped[int] = mapped_column(ForeignKey("child_profiles.id"))

    video_id: Mapped[int] = mapped_column(ForeignKey("video_items.id"))

    watched_seconds: Mapped[int] = mapped_column(Integer)

    quiz_passed: Mapped[bool] = mapped_column(Boolean, default=False)

    attempts_count: Mapped[int] = mapped_column(Integer, default=1)

    timestamp: Mapped[datetime] = mapped_column(DateTime, default=datetime.utcnow)

    child: Mapped["ChildProfile"] = relationship(back_populates="watch_logs")

```

Pydantic Schemas для LLM и API (**app/schemas/moderation.py**)

```

from pydantic import BaseModel, Field

from typing import List, Optional, Literal

class QuizOptionSchema(BaseModel):

    label: str = Field(description="Короткий текст или текстовое описание варианта")

    emoji: str = Field(description="Крупный понятный эмодзи-иллюстрация для кнопки")

```

```
is_correct: bool = Field(description="Флаг правильного ответа")

class GeneratedQuizSchema(BaseModel):

    question_voice_text: str = Field(description="Текст вопроса, который будет озвучен
ребенку голосом через TTS")

    options: List[QuizOptionSchema] = Field(min_length=2, max_length=4,
description="Варианты ответов (2 для возраста 2 года, 3-4 для возраста 5 лет)")

    correct_option_index: int = Field(ge=0, le=3)

class VideoModerationResultSchema(BaseModel):

    is_approved: bool = Field(description="True только для строго спокойного, развивающего
и полезного контента")

    rejection_reason: Optional[str] = Field(default=None, description="Причина отклонения,
если is_approved = False")

    educational_score: int = Field(ge=1, le=10, description="Оценка обучающей ценности
ролика")

    target_age: Literal[2, 5] = Field(description="Целевой возраст ребенка")

    topics: List[str] = Field(description="Ключевые темы и навыки, затрагиваемые в видео")

    summary: str = Field(description="Краткий сжатый пересказ сюжета ролика")

    quizzes: List[GeneratedQuizSchema] = Field(description="Сгенерированные
интерактивные вопросы по содержанию")
```

6. Структура репозитория для Claude Code

kids-edu-ai/

— CLAUDE.md	# Инструкции, контекст и правила генерации для Claude Code
— SPEC.md	# Данная архитектурная спецификация
— docker-compose.yml	# Оркестрация бэкенда, БД и PWA
— .env.example	# Шаблон переменных окружения

```
|— backend/

| |— Dockerfile

| |— requirements.txt    # FastAPI, SQLAlchemy, yt-dlp, google-genai, edge-tts, aiogram

| |— app/

| |   |— main.py        # Точка запуска FastAPI приложения

| |   |— config.py      # Конфигурация через pydantic-settings

| |   |— database.py    # Подключение к БД и сессии SQLAlchemy Async

| |   |— models/        # ORM-модели БД

| |       |— database.py

| |   |— schemas/       # Pydantic v2 схемы (API & LLM)

| |       |— moderation.py

| |   |— api/           # Эндпоинты REST API

| |       |— player.py  # API для детского устройства (получение роликов, отправка
ответов)

| |       |— admin.py   # API для родительской панели

| |   |— services/      # Бизнес-логика системы

| |       |— ingestor.py # Парсинг YouTube/VK через yt-dlp

| |       |— moderator.py # Gemini API Structured Output модерация

| |       |— curriculum.py # Генератор учебного плана и адаптивности

| |       |— tts.py     # Синтез аудиовопросов через edge-tts

| |   |— bot/           # Telegram-бот (aiogram 3.x)

| |       |— main.py

| |       |— handlers.py

|— frontend/
```

```
├── Dockerfile
├── package.json
├── vite.config.ts
├── index.html
├── src/
│   ├── App.tsx          # Главный роутер PWA
│   ├── main.tsx
│   ├── components/
│   │   ├── VideoPlayer.tsx # Защищенный плеер с оверлеем
│   │   └── QuizOverlay.tsx # Модальное окно квиза с озвучкой
│   └── pages/
│       ├── ProfileSelect.tsx # Выбор профиля (2 года / 5 лет)
│       ├── ChildPlayer.tsx  # Главный экран киоска
│       └── ParentAdmin.tsx  # Дашборд статистики
```

7. Пошаговый план разработки (Roadmap)

Этап 1: Инфраструктура и ядро БД (MVP Core)

1. Настройка окружения Python 3.11, FastAPI, SQLAlchemy 2.0 Async.
2. Создание структуры таблиц БД и генерация начальных миграций.
3. Реализация базовых CRUD-эндпоинтов для управления профилями детей (`ChildProfile`).

Этап 2: Контент-пайплайн и ИИ-модерация

1. Реализация `app/services/ingestor.py`: интеграция `yt-dlp` для вытягивания метаданных, субтитров и прямых HLS/MP4 ссылок.
2. Реализация `app/services/moderator.py`: вызов Gemini API со строгим выходом по `VideoModerationResultSchema`.

3. Реализация `app/services/tts.py`: генерация `.mp3` файлов вопросов через `edge-tts` и сохранение в локальный кеш.

Этап 3: Детский Киоск-плеер (Frontend)

1. Разработка SPA/PWA на React + Vite + TailwindCSS.
2. Реализация экрана `ProfileSelect.tsx` с огромными аватарами и крупным тач-интерфейсом.
3. Разработка `VideoPlayer.tsx` с блокировкой кликов по внешним ссылкам.
4. Разработка `QuizOverlay.tsx` с автоматическим проигрыванием TTS-аудио и крупными карточками ответов.

Этап 4: Генератор учебного плана и логика адаптации

1. Реализация алгоритма в `curriculum.py` для формирования темы недели на основе возраста.
2. Интеграция логики расчета процента успешности квизов и автоматического изменения уровня сложности на следующую неделю.

Этап 5: Telegram-бот и развертывание в Kiosk Mode

1. Настройка Telegram-бота на `aiogram 3.x` для отправки отчетов и приемки быстрых команд от родителя.
2. Конфигурация `docker-compose.yml` для автоматического запуска всей системы на WSL2.
3. Настройка Fully Kiosk Browser на Samsung Galaxy S20+ (запрет выхода, автозапуск PWA по локальному IP-адресу).